Criptografia RSA

Aluno: Sinezio Junio Porfirio Araujo • **Matrícula:** 2212130034
Curso: Ciência da Computação • **Turno:** Matutino
Disciplina: Estruturas Matemáticas para Computação

1. Descrição do problema

O programa rsa.py constrói duas chaves a partir de números primos e as utiliza para cifrar e decifrar uma mensagem inteira. Para acompanhar as quatro partes da atividade, considera-se o exemplo p = 5, q = 11, e = 3 e mensagem M = 7. O resultado esperado é recuperar o mesmo inteiro após a decifragem.

2. Partes A e B: dos números primos às chaves

Escolha dos primos. O programa testa possíveis divisores de cada inteiro, a partir de 2, enquanto o quadrado do divisor não ultrapassar o número. Se houver um fator próprio, pelo menos um deles estará nessa faixa. Aceitam-se dois primos distintos: p = 5 e q = 11 fornecem o módulo **n = 5 × 11 = 55**.

Escolha de e. Como n é produto de dois primos distintos, seu totiente é φ(n) = (p − 1)(q − 1) = 4 × 10 = 40. O expoente público precisa satisfazer 1 < e < 40 e ser coprimo com 40. Para e = 3, o procedimento de Euclides produz 40 = 13 × 3 + 1 e 3 = 3 × 1 + 0. O último divisor não nulo é 1; logo, e foi aceito.

Escolha de d. O expoente privado é o inverso de e módulo φ(n). O programa testa candidatos até o produto deixar resto 1. Para d = 27, temos 3 × 27 = 81 = 2 × 40 + 1. Assim, a **chave pública é (3, 55)** e a **chave privada é (27, 55)**. O inverso é calculado módulo 40; a mensagem é cifrada módulo 55.

3. Parte C: cifragem e decifragem

A entrada deve satisfazer 0 ≤ M < n. Para cifrar M = 7, calcula-se $C \equiv 7^3 \pmod{55}$. Partindo de 1, a função multiplica por 7 e guarda somente o resto a cada etapa:

Etapa	Multiplicação	Resto módulo 55
1	1 × 7 = 7	7
2	7 × 7 = 49	49
3	49 × 7 = 343	13, pois 343 = 6 × 55 + 13

Para decifrar C = 13, a mesma função utiliza o expoente privado 27 e multiplica por 13 a cada etapa. O último resto é $\mathbf{13^{27} \bmod 55 = 7}$. Portanto, M′ = M. A redução após cada multiplicação evita construir a potência inteira e preserva o resto final.

4. Parte D: quando aplicar Euler e Fermat

Euler no módulo 55. Como MDC(7, 55) = 1, o Teorema de Euler garante que $7^{40} \equiv 1 \pmod{55}$. A execução confirmou resto 1. Portanto, $7^{41} \equiv 1 \times 7 \equiv 7 \pmod{55}$: o expoente 41 pode ser reduzido a 1 módulo 40. Isso também explica a decifragem do exemplo: $3 \times 27 = 81 = 2 \times 40 + 1$, logo $7^{81} \equiv 7 \pmod{55}$.

Fermat no módulo 5. O módulo 5 é primo e a base 7 não é divisível por 5. O Pequeno Teorema de Fermat dá $7^4 \equiv 1 \pmod{5}$: os restos sucessivos são 2, 4, 3 e 1. Aqui o expoente pode ser reduzido módulo 4. Essa aplicação de Fermat exige um módulo primo; não se pode usar 55, que é composto.

Quando a base não é coprima. Para M = 5, o MDC com 55 é 5. O programa não usa Euler para reduzir o expoente; as multiplicações com redução sucessiva continuam válidas e recuperam M = 5. Para demonstrar Fermat módulo 5, ele utiliza a base q = 11, que não é divisível por 5. A troca de base vale apenas para a demonstração e não altera a mensagem.

5. Testes e resultados

Os seis testes foram executados com o arquivo rsa.py do projeto. Em cada ciclo completo, o valor decifrado coincidiu com a mensagem original.

Teste	Entradas e resultado observado
1. Exemplo principal	p = 5, q = 11, e = 3, M = 7. O programa encontrou d = 27, cifrou 7 como 13 e recuperou 7.
2. Escolha dos primos	A entrada textual “x”, o par (5, 9) e os primos iguais (5, 5) foram rejeitados. O par (2, 3) também foi rejeitado: φ(n) = 2 não admite inteiro e entre 1 e 2. O par (5, 11) foi aceito.
3. Expoente público	Com φ(n) = 40, e = 40 foi rejeitado pelo intervalo; e = 4 foi rejeitado porque MDC(4, 40) = 4. Em seguida, e = 3 foi aceito.
4. Limite da mensagem	Para n = 55, a mensagem 55 foi recusada. Com M = 5, obteve-se C = 15 e M′ = 5, mesmo sem poder aplicar Euler à base 5 no módulo 55.
5. Mensagem zero	Com as mesmas chaves, M = 0 produziu C = 0 e M′ = 0. O limite inferior do intervalo foi aceito.
6. Outro par de primos	p = 7, q = 11, e = 7 e M = 9: n = 77, φ(n) = 60 e d = 43. A saída foi C = 37 e M′ = 9.

Arquivo da atividade: atividade_02_rsa/rsa.py. A verificação de primalidade, o MDC, o inverso e a exponenciação modular são calculados com divisões, produtos e laços.